

JavaScript Interview Questions

– Quick Revision (Most Important Only)

Short version. Only the questions asked most often. For full depth, see `01-javascript-interview-questions.md`.

Table of Contents

1. [var vs let vs const](#)
2. [Hoisting](#)
3. [Closures](#)
4. [this keyword](#)
5. [== vs ===](#)
6. [Promises](#)
7. [async/await](#)
8. [Event Loop](#)
9. [map, filter, reduce, forEach](#)
10. [Spread vs Rest](#)
11. [Destructuring](#)
12. [Promise.all vs allSettled vs race vs any](#)

- 13. [Debounce vs Throttle](#)
 - 14. [Shallow Copy vs Deep Copy](#)
 - 15. [Optional Chaining & Nullish Coalescing](#)
 - 16. [Quick Output-Based Questions](#)
-

1. var vs let vs const

Answer

`var` is function-scoped, gets hoisted with `undefined`, can be redeclared. `let` / `const` are block-scoped. `let` can be reassigned, `const` cannot. We use `const` by default, `let` only when the value will change.

Example

```
if (true) { var x = 1; let y = 2; }
console.log(x); // 1
console.log(y); // ReferenceError
```

Real-world Example

`const` for things you never reassign (navigation object, API response), `let` for a counter or running total.

Follow-up Questions

- Why is `var` avoided in modern code?
 - Can `const` arrays/objects still be mutated internally?
-

2. Hoisting

Answer

JS moves declarations to the top of their scope before running. `var` is hoisted as `undefined`. `let` / `const` are hoisted but stay unusable until their line (Temporal Dead Zone). Function declarations are hoisted fully, so you can call them before they're written.

Example

```
console.log(a); // undefined
var a = 5;
console.log(b); // ReferenceError
let b = 10;
```

Real-world Example

Mostly a trick/output-based interview topic, but understanding it avoids bugs like using a variable

before it's properly set.

Follow-up Questions

- What is the Temporal Dead Zone?
 - Are arrow function expressions hoisted the same way as function declarations?
-

3. Closures

Answer

A closure is when a function remembers variables from where it was created, even after the outer function has finished running.

Example

```
function counter() {  
  let count = 0;  
  return () => ++count;  
}  
const add = counter();  
add(); // 1  
add(); // 2
```

Real-world Example

Used everywhere in hooks – `useState`, custom hooks, debounce functions all rely on closures to remember values across calls/renders.

Follow-up Questions

- Why are closures important for React hooks?
 - What's the classic `var` in a loop closure trap?
-

4. `this` keyword

Answer

`this` depends on how a function is called, not where it's written. Regular functions get `this` from the caller. Arrow functions don't have their own `this` – they use the surrounding scope's `this`. That's why arrow functions are preferred for callbacks.

Example

```
const obj = {
  name: "Asha",
  arrow: () => console.log(this.name), //
  undefined, not obj
  regular() { console.log(this.name); } //
  "Asha"
};
```

Real-world Example

In old class components you had to `.bind(this)` for event handlers. Functional components + hooks mostly remove this problem.

Follow-up Questions

- How does `this` differ in arrow vs regular functions?
 - What does `.bind()` do?
-

5. `==` vs `===`

Answer

`==` compares value only and converts types first (type coercion). `===` compares value and type, no conversion. Always prefer `===` to avoid confusing bugs.

Example

```
1 == "1"; // true
1 === "1"; // false
```

Real-world Example

Checking `response.status === 200` avoids accidental matches if status is ever a string instead of a number.

Follow-up Questions

- Why does `null == undefined` return true?
 - Give an example where `==` causes a real bug.
-

6. Promises

Answer

A Promise represents a value that will be available later. It has 3 states: pending, fulfilled, rejected. Once settled (fulfilled/rejected), it can't change again.

Example

```
fetch("/api")
  .then((res) => res.json())
  .catch((err) => console.log(err));
```

Real-world Example

Every Axios API call (login, fetch list, upload) returns a Promise.

Follow-up Questions

- What is Promise chaining?
 - What happens if you skip `.catch()` ?
-

7. async/await

Answer

A cleaner syntax over Promises — lets async code read like normal step-by-step code. `async` functions always return a Promise. `await` pauses until the Promise settles.

Example

```
async function getUser() {
  try {
    const res = await axios.get("/user/1");
    return res.data;
  } catch (e) {
    console.log(e);
  }
}
```

Real-world Example

Standard way to write API calls in React Native — login, fetch profile, submit forms.

Follow-up Questions

- Why use `try/catch` with `async/await`?
 - Difference between `Promise.all` and multiple sequential `await` calls?
-

8. Event Loop

Answer

JS is single-threaded. The **call stack** runs your code. Background work (timers, API calls) is handled outside, and results go into queues — Promises go to the **microtask queue**, `setTimeout` goes to the **macrotask queue**. The **event loop** runs the call stack first, then clears all microtasks, then takes one macrotask. Microtasks always run before macrotasks.

Example

```
console.log("1");
setTimeout(() => console.log("2"), 0);
Promise.resolve().then(() =>
  console.log("3"));
console.log("4");
// Output: 1, 4, 3, 2
```

Real-world Example

Explains why a state update and a `setTimeout` in the same function don't always run in the order you typed them.

Follow-up Questions

- Why do microtasks run before macrotasks?
 - Is `async/await` micro or macrotask?
-

9. map, filter, reduce, forEach

Answer

`map` returns a new array with each item transformed.
`filter` returns a new array with only matching items. `reduce` boils the array down to one value.
`forEach` just runs code per item and returns nothing.

Example

```
const prices = [100, 200, 300];
prices.map(p => p * 1.18);
prices.filter(p => p > 150);
prices.reduce((sum, p) => sum + p, 0);
```

Real-world Example

`map` to render list items, `filter` for in-stock products, `reduce` for cart total.

Follow-up Questions

- Does `map` mutate the original array?
 - Why not use `forEach` when you need a new array?
-

10. Spread vs Rest

Answer

Spread (`...`) spreads items out — used to copy/combine arrays or objects. Rest (`...`) collects items in — used in function params or destructuring to gather remaining values into an array.

Example

```
const arr2 = [...arr1, 4];           // spread
function sum(...nums) { }           // rest
const { name, ...rest } = user;      // rest
```

Real-world Example

Spread for immutable Redux state updates (`{ ...state, loading: true }`); rest for components

forwarding extra props.

Follow-up Questions

- Is spread a shallow or deep copy?
 - Can rest be used anywhere in params, or only at the end?
-

11. Destructuring

Answer

A short way to pull values out of arrays/objects into variables instead of accessing them one by one.

Example

```
const { name, age = 18 } = user;  
const [first, second] = colors;
```

Real-world Example

Used to pull props in components ({ name, avatarUrl }) and `useState` results ([count, setCount]).

Follow-up Questions

- How do you rename a variable while destructuring?
 - Can you destructure nested objects?
-

12. Promise.all vs allSettled vs race vs any

Answer

`all` waits for all to succeed, fails fast if one fails.
`allSettled` waits for all regardless of success/failure, gives each result. `race` returns as soon as the first one finishes (success or fail). `any` returns as soon as the first one succeeds.

Example

```
const [user, orders] = await  
Promise.all([getUser(), getOrders()]);
```

Real-world Example

`Promise.all` to load a dashboard's profile + orders + notifications in parallel instead of one by one.

Follow-up Questions

- When would you use `allSettled` over `all` ?
- Does `Promise.all` actually run things in parallel?

13. Debounce vs Throttle

Answer

Debounce waits until the user **stops** an action for a set time, then runs once. Throttle runs the function **at most once every X ms**, no matter how often it's triggered.

Example

```
function debounce(fn, delay) {  
  let timer;  
  return (...args) => {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn(...args),  
      delay);  
  };  
}
```

Real-world Example

Debounce on a search input (wait until typing stops).
Throttle on scroll events in a FlatList.

Follow-up Questions

- Where would throttle be better than debounce?

- How would you implement debounce with `useRef` in a custom hook?
-

14. Shallow Copy vs Deep Copy

Answer

Shallow copy only copies the first level — nested objects are still shared (same reference). Deep copy copies everything, fully independent.

Example

```
const shallow = { ...original };
shallow.address.city = "X"; // also changes
original.address.city!
```

Real-world Example

Why mutating nested Redux/ `useState` state without proper copying can fail to trigger a re-render.

Follow-up Questions

- How do you deep copy an object in JS?
 - How does Redux Toolkit's Immer help with this?
-

15. Optional Chaining & Nullish Coalescing

Answer

`?.` safely accesses nested properties without crashing if something is `null` / `undefined` — returns `undefined` instead of throwing. `??` gives a fallback value, but only for `null` / `undefined` (unlike `||`, which also triggers on `0`, `""`, `false`).

Example

```
user.profile?.name;  
const price = product.price ?? 0;
```

Real-world Example

Reading API responses safely:

```
response.data?.user?.address?.pincode .
```

Follow-up Questions

- Why is `??` safer than `||` for defaults?
- Can `?.` be used with function calls or array indexes?

16. Quick Output-Based Questions


```
// Q1: var vs let in a loop
for (var i = 0; i < 3; i++) { setTimeout(()
=> console.log(i), 100); }
// Output: 3, 3, 3 (one shared "i")

for (let j = 0; j < 3; j++) { setTimeout(()
=> console.log(j), 100); }
// Output: 0, 1, 2 (new "j" per iteration)
```

```
// Q2: event loop order
console.log("Start");
setTimeout(() => console.log("Timeout"),
0);
Promise.resolve().then(() =>
console.log("Promise"));
console.log("End");
// Output: Start, End, Promise, Timeout
```

[← Back to top](#)